Certificados Digitales

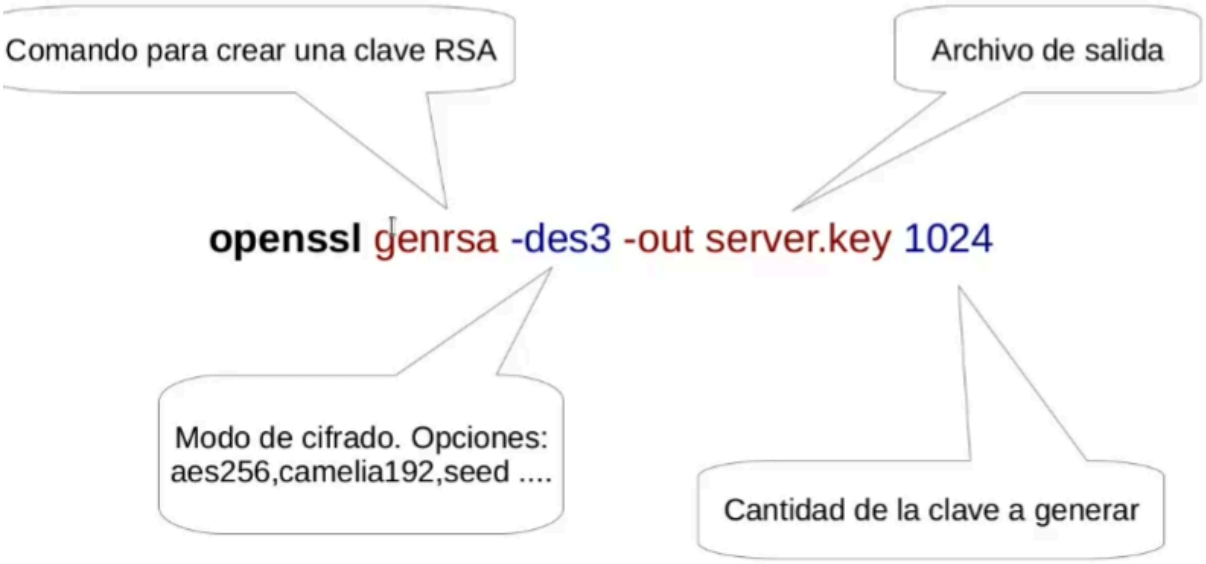
Acrónimos

- BER (Basic Encoding Rules),CER (Canonical Encoding Rules),DER (Distinguished Encoding Rules): Certificado en forma binaria encodeados según la **ITU-T X.690** (Con base en ASN.1)
- CRT (Certificate): En referencia a certificados CER o DER.
- CRL ("Certificate Revocation List"): Lista de certificados revocados
- PEM (Privacy-enhanced Electronic Mail): RFC1421/22/23/24, usualmente aplicado en un certificado DER encodeado en Base64 contenido entre las líneas:
  "-----BEGIN CERTIFICATE-----" y "-----END CERTIFICATE-----"
- JKS (Java Key Store): Formato de repositorios de claves-certificados
- PKCS (Public-Key Cryptography Standards): Estándares definidos por RSA Security para el manejo de información con algoritmos asimétricos. La siguiente tabla enumera y sintetiza estos estándares

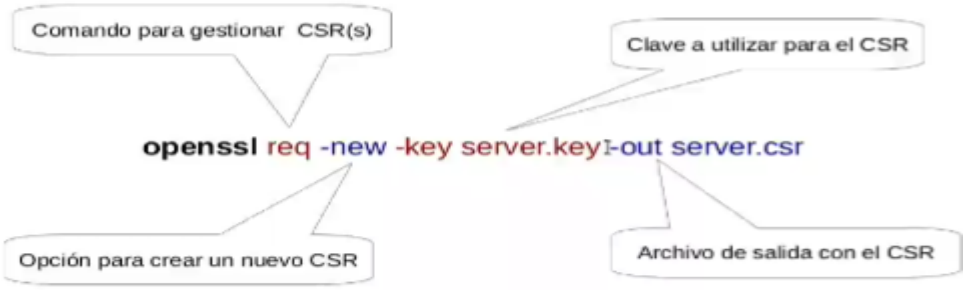
Acrónimo	Contenido	Referencias y detalles
PKCS#1	Estándar criptográfico **RSA**	RFC 3447. Define el formato del cifrado **RSA**
PKCS#2	Obsoleto	Cifrado de resúmenes. Incorporado en **PKCS#1**
PKCS#3	Estándar de intercambio de claves	-
PKCS#4	Obsoleto	Sintaxis de claves. Incorporado en **PKCS#1**
PKCS#5	Estándar de cifrado basado en contraseñas	Padding. **RFC 2898** y **PBKDF2**
PKCS#6	Estándar de sintaxis de certificados	Extendidos X.509 v1. (No en V3)
PKCS#7	Estándar sobre la sintaxis del mensaje criptográfico	**RFC 2315**. Firmar y cifrar mensajes PKI (.p7b, .p7c)
PKCS#8	Estándar sobre la sintaxis de la información de **clave privada**	**RFC 5208**

PKCS#9	Tipos de atributos seleccionados	-
PKCS#10	Estándar de solicitud de certificación	RFC 2986. CSR, solicitud de firma de clave pública
PKCS#11	Interfaz de dispositivo criptográfico (“CryptographicTokenInterface” o cryptoki)	Define un **API** genérico de acceso a dispositivos criptográficos
PKCS#12	Estándar de sintaxis de intercambio de información personal	Formato de repositorio de claves/certificados (.p12, .pfx)
PKCS#13	Estándar de criptografía de curva elíptica	-
PKCS#14	Generación de número pseudo-aleatorios	-
PKCS#15	Estándar de formato de información de dispositivo criptográfico	-

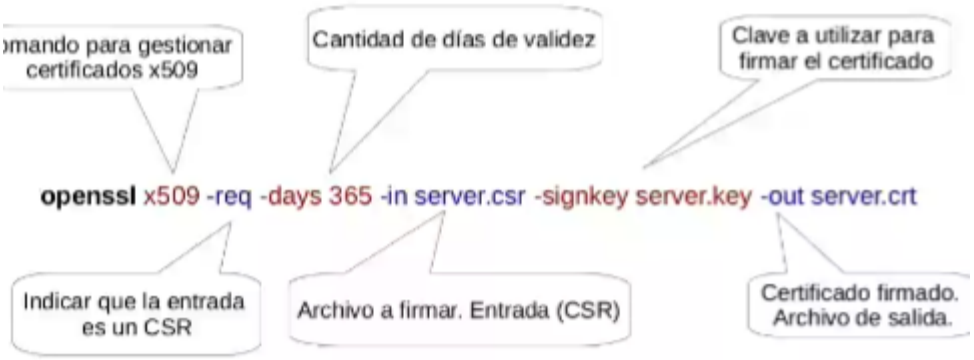
Generar clave RSA



Generar CSR(Certificate Signing Request)



Generar un certificado "Self-Signed"

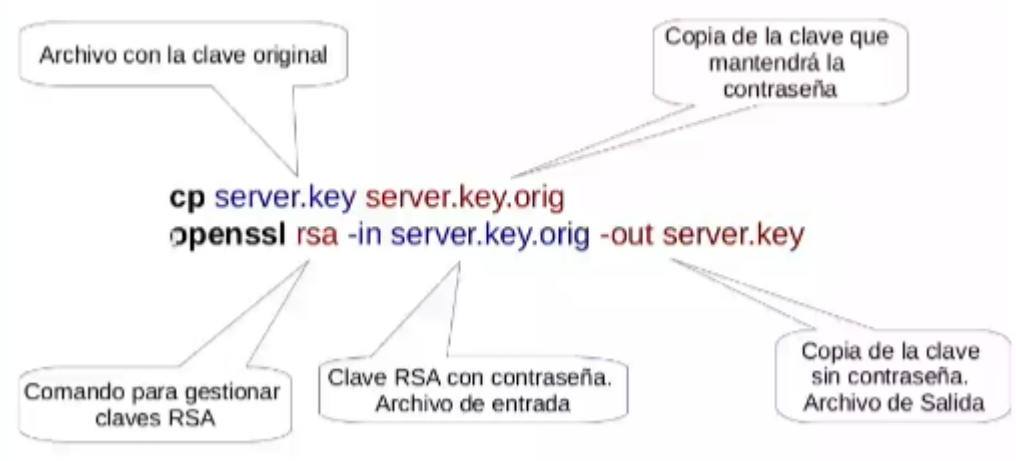


Esta es una alternativa a comprar o solicitar la firma de un tercero confiable (Ej. Verisign)

NOTA: El problema de este certificado es que no te lo puede validar un tercero.

```
OwNGdp7PZaWwHIyazuJ4BfZfsseS19Qkg4VK0erICLgwcDB9Bl+KYSI13DmVY6Bb
J/2gUA==
-----END CERTIFICATE REQUEST-----
dagger@bluegene:~/scaw2025c1$ openssl x509 -req -days 365 -in server.csr
 -signkey server.key -out server.crt
Enter pass phrase for server.key:
Certificate request self-signature ok
subject=C = AR, ST = Buenos Aires, L = Buenos Aires, O = UNLAM, OU = DII
T, CN = scaw.diit.unlam.edu.ar, emailAddress = wureta@unlam.edu.ar
dagger@bluegene:~/scaw2025c1$ ls -l
total 12
-rw-r--r-- 1 dagger dagger 1017 Jun  9 20:26 server.crt
-rw-r--r-- 1 dagger dagger  729 Jun  9 20:19 server.csr
-rw------- 1 dagger dagger 1062 Jun  9 20:13 server.key
dagger@bluegene:~/scaw2025c1$ cat server.crt
-----BEGIN CERTIFICATE-----
MIICwjCCAisCFEvLoHOWC6/MsYddGhGxX+fglEc/MA0GCSqGSIb3DQEBCwUAMIGf
MQswCQYDVQQGEwJBUjEVMBMGA1UECAwMQnVlbm9zIEFpcmVzMRUwEwYDVQQHDAxC
dWVub3MgQWlyZXMxDjAMBgNVBAoMBVVOTEFNMQ0wCwYDVQQLDARESUlUMR8wHQYD
VQQDDBZzY2F3LmRpaXQudW5sYW0uZWR1LmFyMSIwIAYJKoZIhvcNAQkBFhN3dXJl
dGFAdW5sYW0uZWR1LmFyMB4XDTI1MDYwOTIzMjYwMVoXDTI2MDYwOTIzMjYwMVow
gZ8xCzAJBgNVBAYTAkFSMRUwEwYDVQQIDAxCdWVub3MgQWlyZXMxFTATBgNVBAcM
DEJ1ZW5vcyBBaXJlczEOMAwGA1UECgwFVU5MQU0xDTALBgNVBAsMBERJSVQxHzAd
BgNVBAMMFnNjYXcuZGlpdC51bmxhbS51ZHUuYXIxIjAgBgkqhkiG9w0BCQEWE3d1
cmV0YUB1bmxhbS51ZHUuYXIwgZ8wDQYJKoZIhvcNAQEBBQADgY0AMIGJAoGBAMCc
m/HB6RUgS0uX1FGIqw3VyEk5RVI/uckhd9sa/18xbS9tzCD6SZg1XdbVUuCgMkW0
Hk8dBKPt+2cxx5y8gp7DXG3qabgrsVLVGEqHCMq3e9nReQSHNRsBSu9+zdBVp9f0
dzpCdwa0N6cj/P4ETZBeXuIrFzmwpp3JYiy4IB95AgMBAAEwDQYJKoZIhvcNAQEL
BQADgYEABkz9tHEjbRcNo/4dZlRBcA/Y+/KErHvxHslu4WZJgK35hJaYvDosPHSd
QX29ea1tI7hnRM6oVD4mI8iiu0yjlx6OP4i3+dS8yEOPGOf9rJ57MT4YIczS7p/A
ypVFTBcuMhfeujMkdDwSDDgUpDxsuHfppPqKzG+32qb8ffhye08=
-----END CERTIFICATE-----
```

Remover la contraseña de una clave RSA



SSL en Apache Server (httpd)

Instalar la clave privada y su certificado

```
cp server.crt /usr/local/apache/conf/ssl.crt/
cp server.key /usr/local/apache/conf/ssl.key/

Configurar SSL en el Virtual-Host (httpd-vhosts.conf)

SSLEngine on
SSLCertificateFile /usr/local/apache/conf/ssl.crt/server.crt
SSLCertificateKeyFile /usr/local/apache/conf/ssl.key/server.key
SetEnvIf User-Agent "^Mozilla.*$" nounsubscribe ssl-unsubscribe
CustomLog logs/ssl_request_log

%6t %SSLCipherSuite=%{SSL_CIPHER}x
%SSL_PROTOCOL=%{SSL_PROTOCOL}x
%SSL_CIPHER_USEKEYSIZE=%{SSL_CIPHER_USEKEYSIZE}x
%SSL_TLS_COMPRESS=%{SSL_TLS_COMPRESS}x
%SSL_ERROR=%{SSL_ERROR}x
```

Reiniciar el servidor Apache

TLS en Apache Tomcat

Apache Tomcat dispone de dos implementaciones diferentes para utilizar SSL en la comunicación.

Estas son:

- Implementación JSSE, provista por Java desde la versión 1.4.
- Implementación APR, utiliza OpenSSL.

Tomcat dispone de una herramienta para manejo de certificados y repositorios denominada "keytool", adicionalmente cabe destacar que por defecto se maneja el formato "jks" (Java Key Store) en lugar de "PKCS#12".

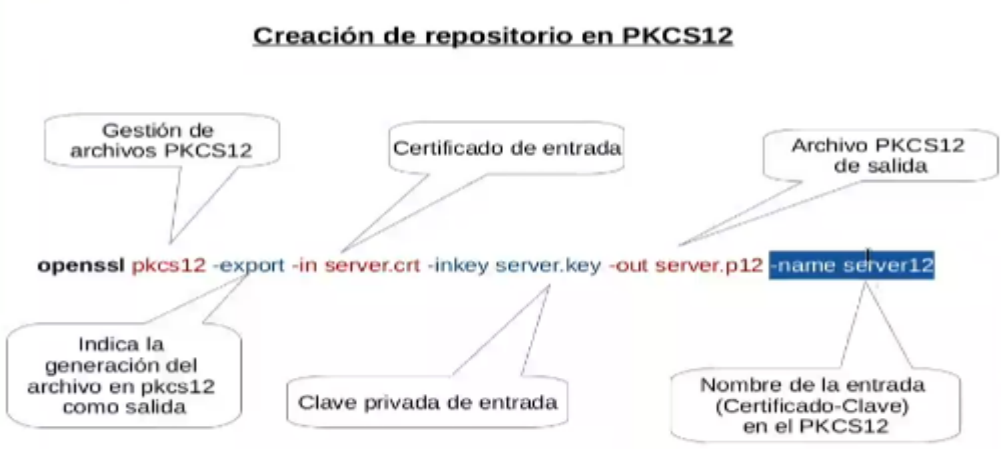
Opcionalmente, el conector a utilizar puede ser seleccionado mediante la configuración del conector correspondiente en el archivo server.xml. A continuación se muestra como realizar esta selección.

```
<!-- Define a blocking Java SSL Coyote HTTP/1.1 Connector on port 8443 -->
<Connector protocol="org.apache.coyote.http11.Http11Protocol"
 port="8443" .../>
<!-- Define a non-blocking Java SSL Coyote HTTP/1.1 Connector on port 8443 -->
<Connector protocol="org.apache.coyote.http11.Http11NioProtocol"
 port="8443" .../>
<!-- Define a APR SSL Coyote HTTP/1.1 Connector on port 8443 -->
<Connector protocol="org.apache.coyote.http11.Http11AprProtocol"
 port="8443" .../>
```

A fin de habilitar el conector correspondiente debemos editar el archivo server.xml con una entrada que configura el mismo indicando los valores específicos de los parámetros y el repositorio de certificados a utilizar.
El siguiente es un ejemplo para la implementación por defecto de Java (JSSE)

```
<Connector port="8443" protocol="HTTP/1.1" SSLEnabled="true"
 maxThreads="150" scheme="https" secure="true"
 clientAuth="false" sslProtocol="TLS"
 keystoreFile="${user.home}/server.p12"
 keystorePass="1234" keystoreType= "PKCS12"
 keyAlias="server12"
```

Creacion de repositorio en PKC12



Generar JKS con clave y certificado auto-firmado

Podemos crear un repositorio JKS con una nueva clave y certificado autofirmado de la siguiente manera.

Certificados digitales en Navegadores Web

A fin de manejar las implementaciones de SSL/TLS los navegadores web actuales necesitan disponer de un repositorio de certificados y funcionalidades para administrarlos. En términos generales encontraremos las siguientes categorías para la administración de los mismos:

- Certificados Propios: Se utilizan para identificar al usuario frente a aplicaciones web que requieran este nivel de autenticación.
- Certificados de Servidores: Se utilizan para que el browser considere a un servidor externo como "Confiable". Algunos browser permiten crear "Excepciones" para cuando un servidor no dispone de un certificado en su base, de esta forma se descarga el mismo y se utiliza de forma temporal.

- Certificados de Autoridades: Se utilizan para validar la firma de un CA sobre un certificado de un servidor y de esta forma considerarlo "Confiable" en base al reconocimiento del CA.

Certificados digitales en Mozilla Firefox

Para acceder a la administración de certificados en Mozilla Firefox debe seguir los siguientes pasos:

- Seleccionar el ítem "Editar" en la barra de menú
- Seleccione la opción de "Preferencias"
- Ir a la sección de "Avanzado"
- Elegir la solapa "Encriptación"
- Dentro de la sección de certificados encontrará el botón "Ver Certificados", el mismo permite acceder a las siguientes categorías con posibilidad de gestión:
    - ☐ Sus Certificados (PKCS#12)
    - ☐ Personas (DER,CER,CRT,PEM)
    - ☐ Servidores (DER,CER,CRT,PEM)
    - ☐ Autoridades (DER,CER,CRT,PEM)
    - ☐ Otros (Dispositivos de seguridad)

Certificados digitales en Google Chrome

Para acceder a la administración de certificados en Google Chrome debe seguir los siguientes pasos:

- Haz clic en el menú de Chrome situado en la barra de herramientas del navegador. Selecciona Configuración.
- Haz clic en la opción para mostrar la configuración avanzada.
- Dentro de la sección de "HTTPS/SSL" encontrará el botón "Administrar Certificados", el mismo permite acceder a las siguientes categorías con posibilidad de gestión:
    - ☐ Sus Certificados (PKCS#12)
    - ☐ Servidores (DER,CRT,PEM,PKCS#7/p7c)
    - ☐ Autoridades (DER,CRT,PEM,PKCS#7/p7c)
    - ☐ Otros (Dispositivos de seguridad)

Su finalidad se vincula a los siguientes puntos:

- Identificar incidentes de seguridad
- Monitorear violaciones a las políticas
- Asistir en controles de no-repudio

- Proveer información sobre problemas o situaciones atípicas
- Contribuir con información específica para la investigación de incidentes que no pueda obtenerse de otras fuentes
- Contribuir con la defensa ante vulnerabilidades y exploits mediante de la detección de ataques

Logging

¿Dónde registrar?

Sistemas de archivos, almacenamiento en la nube, bases de datos SQL y NoSQL

¿Qúe registrar?

Fallos de validación, autenticación, autorización, anomalías, fallas de aplicación, eventos legales...

¿Qúe NO registrar?

Código fuente, identificadores de sesión, credenciales y tokens de acceso, claves de cifrado, SPI

Enmascaramiento de datos:

Es el proceso mediante el cual se reemplazan los datos sensibles de un sistema con el objetivo de proteger esta información confidencial ante situaciones que escapen al control sobre la misma.

Según GDPR, *"seudonimización"*, es el tratamiento de datos personales de manera tal que ya no puedan atribuirse a un interesado sin utilizar información adicional, siempre que dicha información adicional figure por separado y esté sujeta a medidas técnicas y organizativas destinadas a garantizar que los datos personales no se atribuyan a una persona física identificada o identificable.

hay procesos para facilitar el enmascaramiento ,en un entorno real hay que hacer primero el proceso de enmascaramiento y después trabajar con los datos

Debilidades asociadas:

- **CWE-117:** Improper Output Neutralization for Logs
- **CWE-223:** Omission of Security-relevant Information
- **CWE-532:** Insertion of Sensitive Information into Log File
- **CWE-778:**Insufficient Logging

Validación de entrada:

La debilidad de seguridad más común en aplicaciones web es la falta de validación apropiada de las entradas del cliente o del entorno. Esta debilidad lleva a casi todas las principales vulnerabilidades en las aplicaciones web, tales como inyección a intérprete, ataques locales/Unicode, ataques al sistema de archivos y desbordamientos de memoria.

Nunca se debe confiar en los datos introducidos por el cliente, ya que tiene todas las posibilidades de manipular los datos.

La validación debe cubrir los aspectos **semánticos** como **sintácticos** de la información ingresada.
Se sugiere implementar una metodología de validación de tipo "White List" para solo validar el ingreso de datos permitidos.
Para este fin se debe describir el tipo válido de datos, se sugiere implementarlo mediante listas de valores o expresiones regulares.

Expresiones regulares de validación de entrada por tipos:

URL:

```
^(((https?|ftp?s?|gopher|telnet|nntp)://)(mailto:|news:))(%[0-9A-Fa-f]{2}|-(|!~*'/?:@&=+$,A-Za-z0-9])+)(.[.!?/?:,;[]:|abcknmpca-z0-9]+)?$
```

IP:

```
^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?).(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$
```

EMAIL:

```
^[a-zA-Z0-9&-]+(?:.[a-zA-Z0-9_+-]+)*(@:[a-zA-Z0-9]+.[a-zA-Z]{2,7})$
```

NÚMEROS:

```
^(cero|uno|dos|tres|cuatro|cinco|seis|siete|ocho|nueve)$
```

Autenticación en la Web:

Su objetivo es proveer servicios de autenticación segura a las aplicaciones Web, mediante:

- Vinculando una unidad del sistema a un usuario individual mediante el uso de una credencial.
- Proveyendo controles de autenticación razonables de acuerdo al riesgo de la aplicación.

- Denegando el acceso a atacantes que usan varios métodos para atacar el sistema de autenticación.

Cuando hablamos de credenciales hablamos de un conjunto más amplio que de usuario y contraseña.

cuando subo la afectó a la seguridad y disponibilidad.

Autodefensa cuando la app detecta una anomalía y hace una contención.

Es muy perjudicial bloquear el acceso total a un usuario ,en el caso de loguearse si se equivoca 3 veces se lo bloquea temporalmente para que después lo vuelva a intentar,si se vuelve a equivocar si incrementa el tiempo del bloqueo temporal ,esto para evitar ataques a fuerza bruta.

Se recomienda que si un usuario se crea una cuenta su nombre es por ejemplo Walterureta y viene otro walter ureta ,lo ideal no debería dejar que se cree el segundo usuario ,por que seria 2 walter ureta

Consideraciones generales:

- La autenticación es sólo tan fuerte como sus procesos de administración de usuarios
- Use la forma más apropiada de autenticación adecuada para su clasificación de bienes
- Re-autenticar al usuario para transacciones de alto valor y acceso a áreas protegidas
- Autenticar la transacción, no el usuario
- Las contraseñas son trivialmente rotas y no son adecuadas para sistemas de alto valor

Buenas prácticas:

1. **User IDs:** Verificar que **NO** sean case sensitive, para evitar confusiones del tipo “juan” o “Juan”.
2. **Fortaleza de contraseñas:** su longitud mínima debería ser de al menos 10 caracteres y la máxima no inferior a 20, con recomendación de 128 caracteres. Evaluar su complejidad.
3. **Implementar métodos seguros de recuperación:** Con canales externos o diversos puntos de información no selectivos (preguntas de seguridad, de texto libre).
4. **Almacenar contraseñas de forma segura:** con un soporte criptográfico adecuado.
5. **Transmitir contraseñas solo sobre TLS:** la página de logueo y todas las posteriores deben operar bajo TLS.

6. **Solicitar re-autenticación:** Para evitar ataques de CSRF y Hijacking de sesión se debe solicitar autenticación nuevamente antes de realizar operaciones sensibles.

NOTA: Es normal que tengamos rotación de claves,a veces la información no viaja por canales seguros y una forma de que sea seguro temporalmente es el TLS.

Autenticación en la Web

Métodos de protección ante ataques de automatización:

- MFA o Autenticación de factor múltiple
- Bloqueo de cuenta
- CAPTCHA
- Preguntas de seguridad o palabras a memorables

Brute Force: Probar múltiples valores de contraseñas de un diccionario contra para una única cuenta.
Credential Stuffing: Probar combinaciones usuario-contraseña obtenidas de otras brechas de seguridad.
Password Spraying: Probar una contraseña débil sobre un gran número de cuentas.

Contraseñas de un solo uso

OTP "One Time Password" o "Contraseña de un solo uso": corresponde a un valor confidencial que no puede ser reutilizado.

HOTP (Algoritmo de generación de "Contraseñas de un solo uso basadas en HMAC"): RFC-4226

TOTP (Time-based One Time Password o Contraseña de un solo uso basada en tiempo): Es un valor confidencial que no puede ser reutilizado y que además cuenta con un tiempo de vida acotado. RFC-6238.

Técnicas de autenticación de Usuarios

1. Autenticación básica y segura (HTTP-Basic, HTTP-Digest).
2. Autenticación basada en formularios (Usuario-Contraseña).
3. Autenticación integrada (ISS, ASP.NET, Active Directory).
4. Autenticación basada en certificado (x509).
5. Autenticación fuerte o de factor múltiple (Algo que sabes, algo que tienes, algo que eres, tu ubicación).

3 las app se apoya en un sistema central para dar el servicio

4 cada cliente debe configurar el certificado para acceder al sistemas ,seria un paso mas

Autenticacion en la web

- Re uso de contraseñas
- Autenticación sin contraseñas

	OAuth2	OpenID	SAML
Formato del token	JSON o SAML2	JSON	XML
Autorización	Sí	No	Sí
Autentificación	Sí	Sí	Sí
Versión	OAuth2 (2005)	OpenClient (2006)	SAML 2.0 (2001)
Notas	Basado en TLS	-	-
Aplicación	Autorización de API	SSO (Consumidor)	SSO (Empresarial)

Nota: OpenID solo sirve para autenticacion

JWT – JSON Web Token: Es un estándar abierto (rfc7519) utilizado para transmitir de forma segura información en formato JSON. Está compuesto por tres partes: el encabezado que especifica el tipo de token y el algoritmo de firma, la carga útil que contiene los datos relevantes y la firma que verifica la integridad del token. Los tokens JWT se utilizan en aplicaciones web y servicios API para la autenticación y autorización, ya que son seguros, compactos y autocontenidos. Son portables y se pueden utilizar en diferentes sistemas siempre y cuando se comparta la clave necesaria para verificar la firma del token. Esto permite la interoperabilidad entre servicios y aplicaciones.

Autenticación en la Web (Contraseña olvidada)

- Utilizar un mensaje único ya sea que la cuenta exista o no.
- Asegurar que el tiempo de respuesta al usuario sea uniforme.

- Utilizar otro canal para informar el método de re-esteo de la contraseña.
- Utilizar tokens con URL s para una implementación simple y rápida.
- Asegurar que los tokens sean aleatorios, de longitud que provea resistencia a fuerza-bruta, almacenamiento seguro, con tiempo de expiración y de un solo uso.
- No alterar la cuenta del usuario hasta la presentación del token.

Autorización en la Web

Sus objetivos son:

- Asegurar que únicamente usuarios autorizados puedan realizar acciones permitidas con su correspondiente nivel de privilegio.
- Controlar el acceso a recursos protegidos mediante decisiones basadas en el rol o el nivel de privilegio.
- Prevenir ataques de escalada de privilegios, como por ejemplo utilizar funciones administrativas siendo un usuario anónimo o incluso un usuario autenticado.

Métodos de control de acceso:

1. Role Based Access Control (RBAC)
2. Discretionary Access Control (DAC)
3. Mandatory Access Control (MAC)
4. Attribute Based Access Control (ABAC)

Role Based Access Control (RBAC): Las decisiones de acceso se basan en las funciones y responsabilidades de un individuo dentro de la organización o de la base de usuarios. El proceso de definición de las funciones se basa por lo general en el análisis de los objetivos y la estructura de una organización que por lo general está relacionada con la política de seguridad. Por ejemplo, en una organización médica, los diferentes roles de los usuarios pueden incluir aquellos como médico, enfermera, asistente, enfermera, pacientes, etc. Estos miembros requieren diferentes niveles de acceso para llevar a cabo sus funciones, sino también los tipos de las transacciones Web y su contexto permite variar mucho dependiendo de la política de seguridad y los reglamentos pertinentes (HIPAA, Gramm-Leach-Bliley, etc.)

Discretionary Access Control (DAC): Es un medio para restringir el acceso a la información sobre la base de la identidad de los usuarios y/o la pertenencia a ciertos grupos. Decisiones de acceso se basan normalmente en las autorizaciones concedidas a un usuario basándose en las credenciales que presentó en el momento de la autenticación (nombre de usuario, contraseña, hardware / identificador de software, etc.) En los modelos más típicos del DAC, el propietario de

la información o cualquier recurso es capaz de cambiar los permisos a su discreción (de ahí el nombre). DAC tiene el inconveniente de que los administradores no son capaces de gestionar de forma centralizada los permisos de los archivos / datos almacenados en el servidor web. Por ejemplo, el sistema de archivos de un sistema Unix (rwx).

Mandatory Access Control (MAC): Garantiza que la ejecución de la política de seguridad de la organización no se basa en el cumplimiento del usuario en la aplicación web. MAC asegura la información mediante la asignación de etiquetas de sensibilidad en la información y comparando esto con el nivel de sensibilidad de un usuario está operando a. En general, los mecanismos de control de acceso MAC son más seguras que las DAC todavía teniendo soluciones de compromiso en el rendimiento y la conveniencia de los usuarios. Mecanismos MAC asignan un nivel de seguridad a toda la información, asignar un control de seguridad de cada usuario, y garantizar que todos los usuarios sólo tengan acceso a los datos pertinentes. MAC es generalmente apropiado para sistemas extremadamente seguras como aplicaciones militares seguras multinivel o aplicaciones de datos de misión crítica.

ESTE ES EL MAS JERARQUICO

Attribute Based Access Control (ABAC): Es un sistema de control basado en atributos asignados a cualquier componente del sistema (usuarios, recursos, etc). El mismo establece políticas que utilizan la información de dichos atributos para establecer si un acceso está permitido, de esta forma se establece un sistema de control basado en relaciones. Los atributos son del tipo clave-valor, generalmente se utiliza un lenguaje de soporte como XACML para definir las reglas de acceso. Este modelo es muy utilizado en las plataformas actuales de Cloud. Ejemplo: "*Los usuarios pueden acceder a los servicios del proyectoA si pertenecen al grupo de DesarrolloA*". Aquí vemos que la política requiere tres atributos: rol:usuario, servicio:proyectoA, grupo:desarrolloA.

Nota: No necesariamente está obligado a utilizar el mismo tipo de acceso.El discrecional lo tenes en la parte de linux.

Buenas prácticas de implementación:

- Codificar el control en la actividad objetivo
- Disponer de un Controlador Centralizado (ACL)
- Utilizar un Control Central de Acceso, en las diferentes capas
- Verificar la política del lado del servidor (Server-side)

Nota: La política se verifica siempre en el lado del servidor

Ataques de control de acceso:

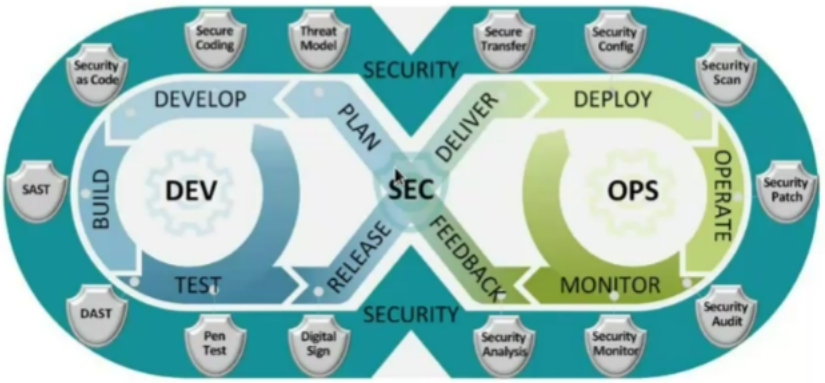
1. **Vertical Access Control Attacks:** Un usuario convencional obtiene accesos superiores o de administrador.
2. **Horizontal Access Control attacks:** Con el mismo rol o nivel el usuario puede acceder a información de otros usuarios.
3. **Business Logic Access Control Attacks:** Usar de una o más actividades para realizar una operación con un resultado no autorizado para ese usuario.

Administración de Usuarios y privilegios

Mejores prácticas

- Cuando se está diseñando aplicaciones, trazar la funcionalidad administrativa fuera y asegurarse de que los controles apropiados de acceso y auditoría están en su lugar.
- Considerar procesos – en algunas ocasiones, todo lo que se requiere es entender cómo los usuarios pueden ser prevenidos de utilizar una característica con la simple falta de acceso.
- Acceso de servicio de asistencia siempre es un término medio – ellos necesitan acceso para ayudar a los clientes, pero no son administradores.(no es una práctica sana por que alguien puede forzar el acceso,no es una buena práctica) .
- Diseñar cuidadosamente la funcionalidad de servicio de asistencia / moderador / soporte al cliente alrededor de una capacidad administrativa limitada y aplicación segregada o acceso.
- Todos los sistemas deberían tener aplicaciones separadas del acceso de los usuarios para los administradores. (Esto no es lo más común) .
- Sistemas de alto valor deberían separar estos sistemas en un servidor separado, que tal vez no sea accesible desde el amplio Internet sin acceso para la administración de redes, como a través del uso de una VPN fuertemente autenticada o desde la red de un centro de operaciones de confianza.

DevSecOps: Es un conjunto de prácticas que combinan el desarrollo de software (Dev), la seguridad (Sec), y las operaciones de tecnología de la información (Ops) para asegurar y acortar el ciclo de vida del desarrollo de software.

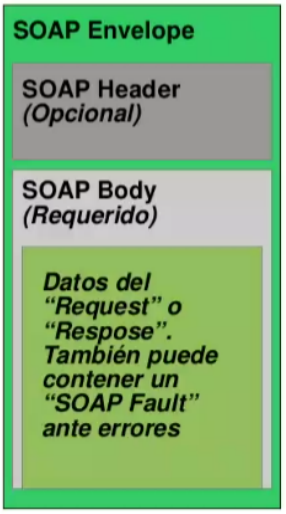


El pen test para que planifique distintos tipos de ataques

Esta tecnologia se usa mucho en el uso de software empresarial

Web Services:

En el nivel más simple, los servicios web pueden ser vistos como aplicaciones web especializadas que difieren principalmente en la capa de presentación. Mientras que las aplicaciones web son típicamente basadas en HTML, los servicios web son basados en XML/SOAP.



Nota: Es muy flexible en cuanto a estructura

Comités de estándares:

- **W3C** (http://www.w3.org): Estándares de esquema XML (XML Schema), SOAP (Simple Object Access Protocol), XML-dsig, XML-enc y WSDL
- **OASIS** (http://www.oasis-open.org): Estándares de WS-Security
- **OASIS** (http://uddi.xml.org): **UDDI** (Universal Description Discovery and Integration)
- **OASIS** (http://saml.xml.org): **SAML** (Security Assertion MarkUp Language)
- Grupo de interoperabilidad de servicios web (WS-I - http://www.ws-i.org/)

Nota: Los servicios Web típicamente representan una interfaz pública funcional, que se llama de forma programática.

Nota: UDDI la gente no se usó mucho en la aparte de búsqueda pero si lo de registro público

Definition WSDL - Parte 1

```
<definitions name="HelloService"
targetNamespace="http://www.examples.com/wsdl/HelloService.wsdl"
xmlns="http://schemas.xmlsoap.org/wsdl/"
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:tns="http://www.examples.com/wsdl/HelloService.wsdl"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<message name="helloRequest">
  <part name="name" type="xsd:string"/>
</message>
<message name="helloResponse">
  <part name="return" type="xsd:string"/>
</message>
<portType name="hello_PortType">
  <operation name="hello">
    <input message="tns:helloRequest"/>
    <output message="tns:helloResponse"/>
  </operation>
</portType>
```

Web Services Definition WSDL - Parte 3

```
<service name="hello_Service">
  <documentation>WSDL for Hello Web Service</documentation>
  <port binding="tns:hello_Binding" name="hello_Port">
```

```
    <soap:address location="http://server.net/hello/"/>
  </port>
</service>
```

Nota: Web services es implementacion en Java con Java EE 5, JAX-WS 2.0

```
package org; import javax.jws.WebService; import javax.jws.WebMethod; import
javax.jws.WebParam;

@WebService(serviceName = "NewWebService") public class NewWebService {
@WebMethod(operationName = "hello") public String hello(@WebParam(name =
"name") String txt) { return "Hola " + txt + "!"; } }
```

Web Services - WS-Security - WSS:El estándar WSS lidia con varias áreas modulares de seguridad, dejando muchos detalles a los llamados documentos perfil. Las áreas principales, ampliamente definidas por el estándar son:

- Maneras de agregar encabezados de seguridad (encabezados WSSE) a los sobres de SOAP
- Adjuntar testigos de seguridad y credenciales al mensaje
- Insertando un estampado de tiempo
- Firmar el mensaje
- Cifrado del mensaje
- Extensibilidad

```
Web Services - WS-Security - WSS

<S11:Envelope xmlns:S11="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:wsse="..."> S11:Header <wsse:Security xmlns:wsse="..."> <xxx:CustomToken
wsud:id="MyID" xmlns:xxx="http://fabrikam123/token"> <FHIUORV.../>
</xxx:CustomToken> ds:Signature ds:SignedInfo <ds:CanonicalizationMethod
Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#"/> <ds:SignatureMethod
Algorithm="http://www.w3.org/2000/09/xmldsig#hmac-sha1"/> <ds:Reference
URI="#MsgBody"> <ds:DigestMethod
Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
ds:DigestValueLyLS0FPI4wPU...</ds:DigestValue> </ds:Reference>
</ds:SignedInfo> ds:SignatureValueDJbchm5gK...</ds:SignatureValue> ds:KeyInfo
wsse:SecurityTokenReference <wsse:Reference URI="#MyID"/>
</wsse:SecurityTokenReference> </ds:KeyInfo> </ds:Signature> </wsse:Security>
</S11:Header> S11:Body <tru:StockSymbol
xmlns:tru="http://fabrikam123.com/payloads"> QQQ </tru:StockSymbol>
</S11:Body> </S11:Envelope>
```

WS-Policy: Describe capacidades y limitaciones de la seguridad, políticas e intermediarios (reglas de seguridad, algoritmos soportados, etc).

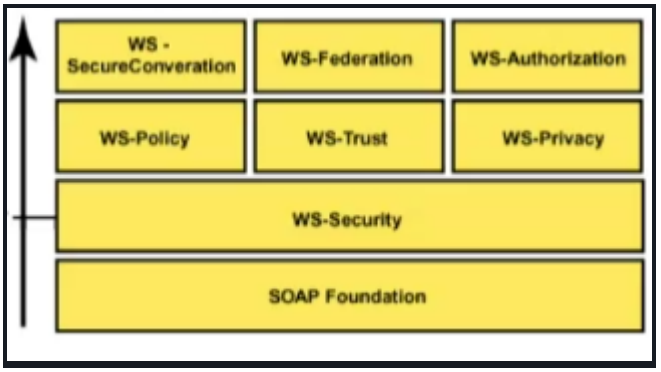
WS-Trust: Describe un framework para facilitar la interoperación de WS en forma segura.

WS-Privacy: Describe el modelo sobre cómo los Web Services manejan las peticiones y preferencias de seguridad.

WS-SecureConversation: Describe cómo manejar y autenticar el intercambio de mensajes, el contexto de seguridad y claves de sesión.

WS-Federation: Describe cómo administrar y manejar las relaciones de confianza entre sistemas federados.

WS-Authorization: Describe cómo administrar la autorización de datos y políticas.informativo (no profundizan).



ReST: Es una técnica de arquitectura para sistemas distribuidos, su nombre es un acrónimo que deriva de "Representational State Transfer". Su origen data del año 2000 siendo definido en una tesis doctoral de Roy Fielding.
Su objetivo fue evitar el uso de métodos complejos como CORBA, RPC y SOAP para interconexión de sistemas; con este fin las aplicaciones ReSTful usan llamados HTTP para las operaciones CRUD (Create/Read/Update/Delete) orientando su diseño a elementos en lugar de operaciones.
Es una técnica de arquitectura para sistemas distribuidos, su nombre es un acrónimo que deriva de *"Representational State Transfer"*. Su origen data del año 2000 siendo definido en una tesis doctoral de Roy Fielding.

Su objetivo fue evitar el uso de métodos complejos como CORBA, RPC y SOAP para interconexión de sistemas; con este fin las aplicaciones ReSTful usan llamados **HTTP** para las operaciones **CRUD (Create/Read/Update/Delete)** orientando su diseño a elementos en lugar de operaciones.

WADL - Web Application Description Language

```
<resources base="http://example.com/widgets">
  <resource path="{widgetId}">
    <param name="customerId" style="query"/>
    <method name="GET">
      <request>
        <param name="verbose" style="query" type="xsd:boolean"/>
        <param name="text" style="query" type="xsd:string"/>
      </request>
      <response status="200">
        <representation mediaType="application/xml" element="yn:ResultSet"/>
      </response>
    </method>
  </resource>
</resources>
)
```